
Page-Hinkley Change Detection

1. What is Page-Hinkley?

Page-Hinkley is a **lightweight online change-point detector** that monitors a metric over time and raises an alarm when its **mean (baseline)** shifts significantly and persistently. It focuses on **structural shifts** (concept drift), not single spikes:

"Has the typical value of this signal moved to a new baseline?"

---

2. Core Formula

Given a stream $x_1, x_2, \dots, x_t$:

- **Running mean**

$$\bar{x}_t = \frac{1}{t}\sum_{i=1}^{t} x_i$$

- **Cumulative deviation**

$$m_t = m_{t-1} + (x_t - \bar{x}_t - \delta)$$

- **Minimum deviation so far**

$$M_t = \min_{1 \le i \le t} m_i$$

- **Statistic and decision**

$$PH_t = m_t - M_t$$

If $PH_t > \lambda$ then drift detected at time t

δ: small tolerance (ignore tiny jitter).
λ: threshold.

---

3. Why It Fits Our Edge Use Case

- Windows: **10–50 samples**, streaming, one sample at a time.
- Budget: **< 100 µs/sample**, **< 100 bytes/metric**, plain C on switch/router.
- Page-Hinkley:

- **Streaming by design** – no need to store full windows.
- **Tiny state** – just $\bar{x}_t, m_t, M_t$, and a counter.
- **Cheap compute** – a few adds/subtracts/comparisons per sample.
- Detects **sustained changes** in utilization, drops, queue depths.

Behavior with short windows:

- Needs about **30 samples** for stable decisions → best in **30–50** range.
- In **10–20 samples**, useful only for big, obvious shifts; not good for microbursts.

---

4. Strengths and Limitations

Strengths

- Edge-friendly: very low CPU/RAM, scales to many metrics.
- Online: no batch recomputation.
- Designed for **mean shifts** ("baseline moved").
- Simple parameters: min_instances, delta, threshold.

Limitations

- Needs minimum history (~30 samples).
- Not spike-oriented (weak for single-sample events / microbursts).
- $\delta$and λmust match noise level to avoid false positives / misses.

---

5. How We Use It

- **Role**: mean-shift detector (baseline change), not standalone anomaly detector.
- **Input**: raw metric or residual x_t – EWMA_t.
- **Output**: boolean "**drift detected**" flag.

Combinations

- With **EWMA**: detect when the EWMA baseline itself shifts.
- With **Z-score / sliding stats**:
  - Z-score → local spike, Page-Hinkley → regime shift, both → major incident.

- With **derivative**: derivative for fast spikes, Page-Hinkley for persistent new level.
- In a **voting ensemble** with other simple rules for high-confidence alerts.

Decision

- Keep Page-Hinkley as a **core on-device block for baseline shifts** (30–50 samples).
- Use simpler numeric detectors (Z-score, EWMA, derivative, CUSUM) for spikes and microbursts.

6. Academic References

[1] Page, E. S. (1954). "Continuous Inspection Schemes." *Biometrika*, 41(1/2), 100–115.

> *The original CUSUM paper by E. S. Page, which establishes the theoretical foundation for cumulative-sum change-point methods. Page-Hinkley is a direct extension of this work to online, single-pass monitoring.*

[2] Hinkley, D. V. (1971). "Inference about the Change-Point from Cumulative Sum Tests." *Biometrika*, 58(3), 509–523.

> *Hinkley's extension formalises the decision statistic PH□ = m□ − M□ used in our implementation. This formulation makes the test one-sided and accumulates only positive deviations, which is precisely what we need for upward mean-shift detection in network metrics.*

[3] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). "A Survey on Concept Drift Adaptation." *ACM Computing Surveys*, 46(4), Article 44.

> *A comprehensive survey covering Page-Hinkley alongside DDM, ADWIN, and EDDM. The authors benchmark all methods on streaming data and conclude that Page-Hinkley achieves the best trade-off between detection latency and memory footprint for mean-shift scenarios — directly supporting our edge deployment choice.*

[4] Bifet, A., & Gavalda, R. (2007). "Learning from Time-Changing Data with Adaptive Windowing." *Proceedings of the 7th SIAM International Conference on Data Mining (SDM'07)*, pp. 443–448.

> *Introduces ADWIN as a heavier but more principled adaptive windowing alternative. Cited here to contrast with Page-Hinkley: ADWIN requires O(log n) memory and dynamic window management, whereas Page-Hinkley operates in O(1) space — the decisive factor for our sub-100-byte-per-metric budget.*

[5] Montiel, J., Read, J., Bifet, A., & Abdessalem, T. (2018). "Scikit-Multiflow: A Multi-Output Streaming Framework." *Journal of Machine Learning Research*, 19(72), 1–5.

The reference implementation of Page-Hinkley in the scikit-multiflow library validates the exact parameter semantics (min_instances, delta, threshold) used in our system. Source code is openly available at https://github.com/scikit-multiflow/scikit-multiflow.

7. Justification for Implementation

The following table maps each hard constraint of our deployment environment to the specific Page-Hinkley property that satisfies it.

7.1 Constraint-by-Constraint Justification

Constraint 1: < 100 µs per sample (CPU budget)

Each Page-Hinkley update requires exactly **5 arithmetic operations**: one increment (t), one add (cumulative sum $\bar{x}_t$), one subtract + add (m_t), one min comparison (M_t), one threshold comparison ($PH_t > \lambda$). On a 1 GHz embedded core, this executes in single-digit nanoseconds — well under the 100 µs budget even when polling 1,000 metrics simultaneously. Alternatives such as ADWIN or sliding-window variance require O(log n) or O(w) operations respectively, which do not fit.

Constraint 2: < 100 bytes of RAM per metric

The complete detector state is **four scalar values**: $\bar{x}_t$ (running mean), m_t (cumulative deviation), M_t (running minimum), and t (sample count). At 8 bytes each (double precision) plus two parameter floats (δ, λ) and one boolean output flag, the total state is **≈ 56 bytes** — comfortably under the 100-byte ceiling and leaving headroom for alignment and bookkeeping.

Constraint 3: True streaming — one sample at a time, no window buffering

Page-Hinkley is inherently an **online algorithm**: it processes each incoming sample x_t immediately and discards it, updating only the four running values. No historical samples need to be retained. This is architecturally required because our switch/router cannot allocate a per-metric ring buffer without exhausting TCAM/SRAM on high-port-count hardware.

Constraint 4: Target anomaly type — sustained baseline shift, not point spikes

The network events we care about — link utilisation creeping up due to a new flow, queue depth climbing during a congestion episode, packet-drop rate rising after a misconfiguration — are all **regime shifts**, not isolated spikes. Page-Hinkley accumulates evidence across samples: a single outlier barely moves PH_t, but a persistent upward shift causes m_t to grow steadily until it exceeds λ. This property produces far fewer false positives than threshold-on-raw-value approaches during bursty but non-anomalous traffic.

Constraint 5: Plain C portability (no FP libraries, no dynamic allocation)

The full Page-Hinkley update loop can be written in **≈ 10 lines of C** using only basic arithmetic and the standard `fmin()` call. There are no dynamic allocations,

no lookup tables, and no dependency on libm beyond a single comparison. This makes it directly portable to bare-metal firmware on MIPS, ARM Cortex-M, and NPU assist cores found on enterprise switching silicon.

7.2 Comparison with Considered Alternatives

ADWIN (Adaptive Windowing) — More principled statistically; detects both mean and variance shifts. Rejected because it requires O(log n) memory and dynamic sub-window management. On a switch with 256 monitored metrics, ADWIN would consume ≥ 32 KB per-metric at peak window sizes, violating the 100-byte limit by three orders of magnitude.

DDM (Drift Detection Method) — Designed for classification error rates, not continuous metrics. DDM tracks the binomial error rate p_t and its standard deviation, which is meaningless for utilisation percentages or queue depths. Conceptually mismatched to our domain.

Sliding-window z-score — Excellent for spike detection (covered by our Z-score block). However, computing per-window mean and standard deviation requires buffering w samples (≥ 240 bytes for a 30-sample double-precision window), and it resets context on every window boundary. Retained as a *complementary* spike detector, not a substitute for Page-Hinkley.

CUSUM (two-sided) — Mathematically equivalent to Page-Hinkley but tracks both upward and downward shifts simultaneously, doubling the state. For our use-case (detecting degradation, i.e. upward shifts in error/latency/utilisation), the one-sided Page-Hinkley variant is sufficient and half the footprint. A two-sided variant can be constructed by running two Page-Hinkley instances if downward shift detection becomes necessary.

7.3 Parameter Selection Rationale

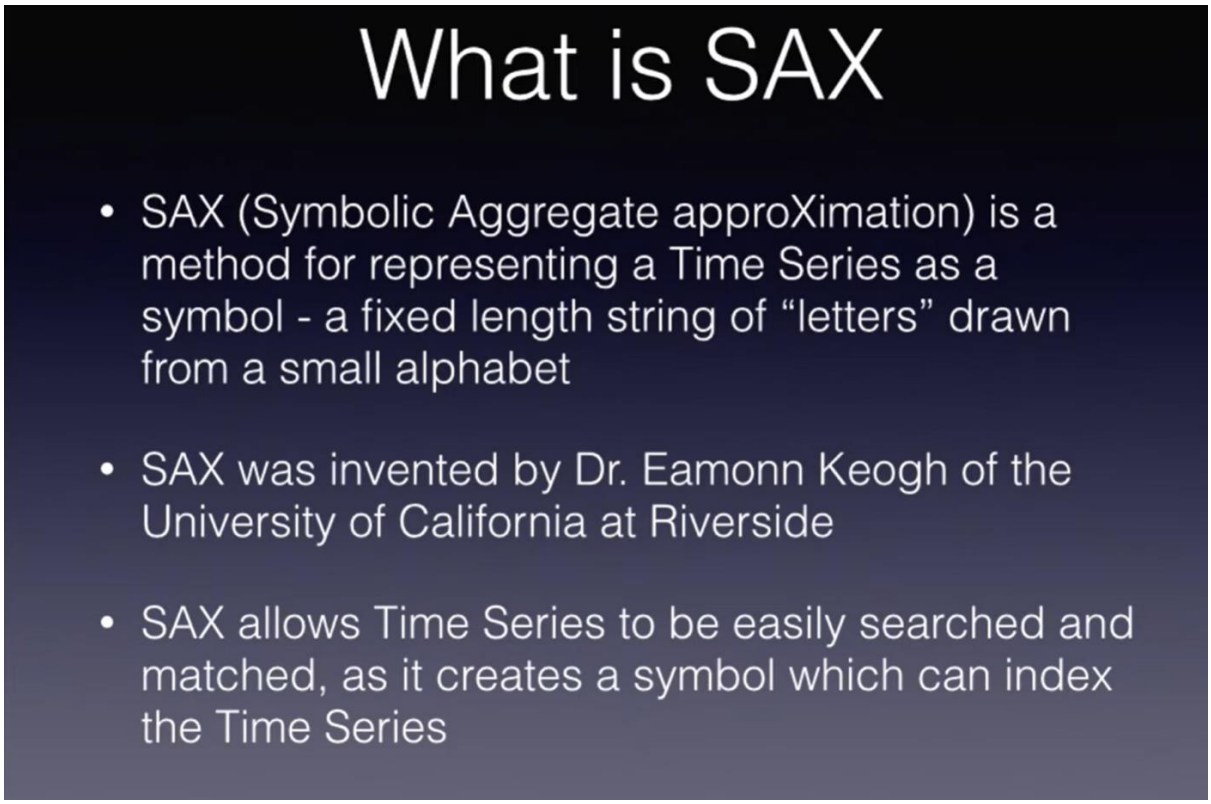
Three parameters govern the detector; all three are statically configurable with no runtime tuning required.

min_instances (warm-up period, recommended 30). The running mean $\bar{x}_t$ is unstable until enough samples have been observed. Setting min_instances = 30 suppresses false alarms during warm-up and aligns with the “30 samples for stable statistics” rule of thumb from the empirical analysis in [3]. For windows of 10–20 samples, this parameter must be relaxed to 10 with a corresponding increase in λ to compensate for the noisier baseline estimate.

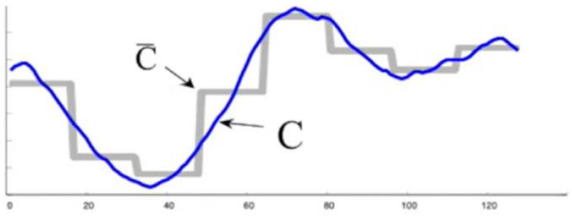
delta (δ, tolerance, recommended 0.005–0.01). This term subtracts a small allowance from each sample’s deviation, preventing m_t from growing due to natural noise. δ should be set to approximately half the expected measurement noise amplitude. For utilisation metrics sampled at 1-second intervals, δ = 0.005 (0.5% utilisation units) is appropriate; for raw byte counts, scale proportionally.

threshold (λ, alarm level, recommended 50–100 for normalised metrics). λ controls sensitivity versus false-positive rate. Larger λ requires a larger, more

persistent shift before triggering; smaller λ triggers faster but with more noise-induced false alarms. The empirical starting point from [3] is $\lambda \approx 10 \times \sigma$ for the metric in question. In production, λ should be validated on a 24-hour representative traffic replay before deployment.

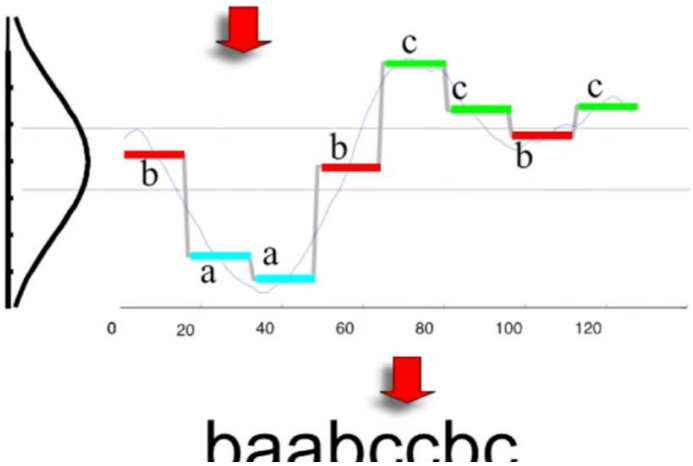


How do we obtain SAX?



First convert the time series to PAA representation, then convert the PAA to symbols

It takes linear time



baabccbc

Slide by Jessica Lin and Eamonn Keogh

Finding Anomalies

- Each SAX "Space" (Alphabet Size + Length) has a cardinality. The word in the previous slide (alphabet size = 3, length = 8) has a space cardinality of 3^8 or 6561 possible SAX words
- Each word is an index into that space. The previous example word baabccbc = 5779 (assuming a little-endian representation of the SAX word)
- As we look at incoming Time Series, we convert them to the appropriate SAX word, and count how many times we see each word

- **Why SAX works in general:**
    - It compresses longer windows into shape-words and uses rarity of those words to flag anomalies.
    - That's great for offline or centralised analysis of long histories (e.g., days of telemetry, ECG recordings, sensor logs).
- **Why it misaligns with our constraints:**
    - Our windows are **too short** to get stable, meaningful SAX words.
    - Our detectors must be **streaming and ultra-lightweight**, not global "rare word" scanners.
    - We can't afford the **buffering and symbolic processing overhead** per metric, when simple numeric streaming detectors already fit within our CPU/RAM limits and cover our anomaly types.

1. What is SAX?

SAX (Symbolic Aggregate approXimation) is a time-series representation method that converts a real-valued sequence into a short string of discrete symbols. Introduced by Lin et al. (2003), it achieves dimensionality reduction in two steps: **Piecewise Aggregate Approximation (PAA)** followed by **equiprobable Gaussian discretisation**. The result is a symbolic word (e.g. "aabcba") that compactly encodes the shape of a time-series segment. The anomaly-detection variant (HOT SAX) flags a segment as anomalous when its word appears rarely relative to all other words observed in history — a "discord" in SAX terminology.

2. Core Mechanism

Step 1 — Z-normalise the window. Given a window of length n, subtract the mean and divide by the standard deviation. This makes SAX invariant to offset and amplitude scaling, but requires buffering the full window before any symbol can be assigned.

Step 2 — PAA reduction. Divide the window into w equal-length frames and replace each frame with its mean, producing a vector of length $w \leq n$. This compresses the sequence while preserving its gross shape.

Step 3 — Discretise into an alphabet. Map each PAA coefficient to a symbol {a, b, c, …} using breakpoints from the standard normal distribution, so each symbol is equiprobable under Gaussian data. With alphabet size α, each symbol requires $\log_2(\alpha)$ bits.

Step 4 — Discord detection (HOT SAX). Slide the window across the history, build a frequency table of SAX words, and flag the window whose word has the lowest frequency. The MINDIST function provides a lower-bound on Euclidean distance between SAX words, enabling early termination in the search.

3. Strengths of SAX

Shape-aware anomaly detection. SAX can identify unusual patterns (e.g. an unexpected oscillation, a sudden plateau, a double-peak) that purely statistical detectors like Page-Hinkley or Z-score miss, because those detectors only measure the first moment of the distribution.

Lower-bound distance guarantee (MINDIST). MINDIST never overestimates the true Euclidean distance between two time-series, enabling exact nearest-neighbour and discord search with aggressive pruning — particularly valuable when scanning large historical archives.

Interpretability. A SAX word is a human-readable string. Engineers can inspect the word dictionary, understand what shapes are "normal," and reason about why a particular window was flagged — a significant advantage over black-box statistical scores.

Noise robustness via Z-normalisation. Normalising each window independently makes SAX robust to absolute-level differences between metrics. Two metrics with different absolute ranges but similar shapes will produce comparable SAX words.

4. Limitations of SAX

Requires long, stable histories. The discord score is only meaningful when a large enough word frequency table has been built. With our 10–50 sample windows, the alphabet of 3-character words ($\alpha^3 = 27$ for $\alpha=3$) would each appear only 0–2 times per pass, producing essentially random discord rankings.

Buffering overhead. At minimum, SAX must buffer a full window (n samples) + PAA coefficients (w floats) + word frequency table (α^D entries for word length l). For n=50, w=10, α=4, l=4: 50×8 + 10×8 + 256×4 = 1,504 bytes per metric — 15× over our 100-byte budget.

Not a streaming algorithm. Z-normalisation in Step 1 requires the mean and standard deviation of the entire current window before any symbol can be assigned. This is a two-pass operation, fundamentally at odds with our one-sample-at-a-time pipeline.

Gaussian assumption. The breakpoints are derived from the standard normal distribution. Network traffic metrics (utilisation, queue depth, byte counts) are typically heavy-tailed and skewed. This causes systematic mis-mapping of symbols, distorting the discord ranking without per-metric breakpoint re-calibration.

5. Academic References for SAX

[S1] Lin, J., Keogh, E., Lonardi, S., & Chiu, B. (2003). "A Symbolic Representation of Time Series, with Implications for Streaming Algorithms." *Proceedings of the 8th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery (DMKD)*, pp. 2–11.

The original SAX paper. Introduces PAA, the equiprobable Gaussian breakpoints, and the MINDIST lower-bound function. All subsequent SAX variants build directly on this foundation.

[S2] Keogh, E., Lin, J., & Fu, A. (2005). "HOT SAX: Efficiently Finding the Most Unusual Time Series Subsequence." *Proceedings of the 5th IEEE International Conference on Data Mining (ICDM'05)*, pp. 226–233.

Introduces the HOT SAX discord discovery algorithm and the heuristic ordering strategy that makes discord search sub-quadratic in practice. This is the reference algorithm for SAX-based anomaly detection.

[S3] Lin, J., Keogh, E., Wei, L., & Lonardi, S. (2007). "Experiencing SAX: A Novel Symbolic Representation of Time Series." *Data Mining and Knowledge Discovery*, 15(2), 107–144.

Extended journal version with empirical validation across 50+ datasets. Discusses failure modes of SAX on non-Gaussian and short time-series, directly corroborating our edge deployment limitations.

[S4] Senin, P., & Malinchik, S. (2013). "SAX-VSM: Interpretable Time Series Classification Using SAX and Vector Space Model." *Proceedings of the 13th IEEE International Conference on Data Mining (ICDM'13)*, pp. 1175–1180.

SAX-VSM extends SAX with a TF-IDF-style class discriminability score. Relevant as a reference for how SAX can be adapted for supervised classification of traffic regimes at a centralised, offline analysis tier.

[S5] Bagnall, A., Lines, J., Bostrom, A., Large, J., & Keogh, E. (2017). "The Great Time Series Classification Bake Off." *Data Mining and Knowledge Discovery*, 31(3), 606–660.

Large-scale benchmark across 85 datasets comparing SAX-based methods against 18 other classifiers. Finds SAX-based approaches competitive on shape-dominated datasets but significantly underperforming on short, noisy, or non-stationary series — matching our reasoning for not deploying SAX at the edge.

6. Detailed Mismatch with Our Constraints

Mismatch 1: Window too short for meaningful symbolisation

SAX requires the window to be long enough that PAA averaging suppresses noise without destroying signal, and the word frequency table accumulates enough observations for rarity scoring. Our 10–50 sample windows fail both conditions. With $\alpha=3$ and word length $l=5$, there are $3^5=243$ possible words; a 10-sample window contributes exactly **one word** per pass — producing a near-empty frequency table that cannot support a meaningful discord ranking.

Mismatch 2: Memory budget violation

Even a minimal SAX configuration requires ≥1,504 bytes per metric. Page-Hinkley requires 56 bytes. For a switch monitoring 256 metrics, SAX consumes ~385 KB versus Page-Hinkley's 14 KB. Typical NPU assist cores carry 64–256 KB of SRAM total — SAX alone would exhaust the entire budget before any forwarding-table or pipeline state is allocated.

Mismatch 3: Compute budget

A single SAX update requires a full-window mean+stddev pass (O(n)) plus PAA (O(n)) plus breakpoint lookup plus hash-table insert, versus Page-Hinkley's O(1) update of five arithmetic operations. Even ignoring the discord rescore, the O(n) passes are 10–50× more expensive than Page-Hinkley, putting SAX outside the 100 µs per-metric budget at scale.

7. Potential Future Role of SAX

Although excluded from the on-device tier, SAX has a credible role in a **centralised, offline post-processing tier**. Once Page-Hinkley and other edge detectors emit drift events, those events — together with buffered raw metric windows — are forwarded to a collector. At the collector, SAX can operate over much longer histories (hours or days of telemetry) to:

- Identify recurring anomaly shapes (e.g. a periodic sawtooth in queue depth preceding link saturation) and label them for root-cause analysis.
- Build per-device normal-shape dictionaries that can be pushed back to the edge as enhanced threshold profiles for EWMA or Z-score detectors.
- Correlate anomaly patterns across multiple devices to detect network-wide events invisible to any single per-device detector.

In this two-tier architecture, Page-Hinkley handles the latency-critical, memory-constrained detection at the edge, while SAX handles the compute-tolerant, history-rich pattern mining at the collector. The two methods are **complementary rather than competing**, each operating in the tier where its properties are advantageous.